

TP d'Analyse Factorielle des Correspondances (AFC)

DUT : GI-SIR-IDIA / ESTBM USMS

Pr. Abdelaziz QAFFOU

Introduction

L'Analyse Factorielle des Correspondances (AFC) est une méthode d'analyse exploratoire de données permettant d'étudier les associations entre les modalités de deux variables qualitatives. Elle s'applique à des tableaux de contingence et permet de visualiser les relations sous forme de cartes factorielles.

1 Données d'exemple

Nous utiliserons le jeu de données **HairEyeColor** disponible dans R, qui recense la couleur des cheveux et des yeux de 592 étudiants.

1.1 Tableau de contingence

Le tableau croise les variables "Cheveux" (4 modalités) et "Yeux" (4 modalités).

	Brun	Bleu	Noisette	Vert
Noir	68	20	15	5
Brun	119	84	54	29
Roux	26	17	14	14
Blond	7	94	10	16

TABLE 1 – Tableau de contingence Couleur des Cheveux & Couleur des Yeux

2 Implémentation en R

2.1 Chargement des données et préparation

```
1 # Chargement des données
2 data(HairEyeColor)
3
4 # Extraction du tableau hommes + femmes
5 hair_eye <- HairEyeColor[, , "Male"] + HairEyeColor[, , "Female"]
6
```

```

7 # Affichage du tableau
8 print(hair_eye)
9
10 #
11 # Hair      Eye
12 #   Black   Brown Blue Hazel Green
13 #   Black   68   20   15    5
14 #   Brown  119   84   54   29
15 #   Red    26   17   14   14
16 #   Blond   7   94   10   16

```

2.2 Réalisation de l'AFC

```

1 # AFC avec FactoMineR
2 library(FactoMineR)
3 library(factoextra)
4
5 # Réalisation de l'AFC
6 afc_result <- CA(hair_eye, graph = FALSE)
7
8 # Valeurs propres (inerties)
9 eig.val <- get_eigenvalue(afc_result)
10 print(eig.val)
11
12 #      eigenvalue variance.percent cumulative.variance.percent
13 # Dim.1  0.20877355          79.241887          79.24189
14 # Dim.2  0.04426338          16.802347          96.04423
15 # Dim.3  0.01039642           3.955774         100.00000
16
17 # Pourcentage d'inertie expliquée
18 cat("Inertie totale:", sum(afc_result$eig[,1]), "\n")
19 cat("Inertie expliquée Dim1:", eig.val[1,2], "%\n")
20 cat("Inertie expliquée Dim2:", eig.val[2,2], "%\n")

```

2.3 Visualisation des résultats

```

1 # Graphique des valeurs propres
2 fviz_screplot(afc_result, addlabels = TRUE) +
3 ggtitle("Éboulis des valeurs propres")
4
5 # Carte factorielle
6 fviz_ca_biplot(afc_result,
7 repel = TRUE,
8 title = "AFC - Carte factorielle",
9 col.row = "blue",
10 col.col = "red")
11
12 # Contributions des lignes et colonnes
13 fviz_ca_row(afc_result, col.row = "contrib",
14 gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
15 repel = TRUE)
16

```

```

17 fviz_ca_col(afc_result, col.col = "contrib",
18 gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
19 repel = TRUE)

```

2.4 Interprétation des résultats

```

1 # Contributions
2 row_contrib <- afc_result$row$contrib
3 col_contrib <- afc_result$col$contrib
4
5 print("Contributions des lignes:")
6 print(row_contrib)
7
8 print("Contributions des colonnes:")
9 print(col_contrib)
10
11 # Cos2 (qualité de représentation)
12 row_cos2 <- afc_result$row$cos2
13 col_cos2 <- afc_result$col$cos2
14
15 # Coordonnées factorielles
16 print("Coordonnées des lignes:")
17 print(afc_result$row$coord)
18
19 print("Coordonnées des colonnes:")
20 print(afc_result$col$coord)

```

3 Implémentation en Python

3.1 Installation et importation des bibliothèques

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.decomposition import PCA
5 from scipy import stats
6 import prince # Pour l'AFC
7 import seaborn as sns
8
9 # Configuration des graphiques
10 plt.style.use('seaborn-v0_8-whitegrid')
11 sns.set_palette("husl")

```

3.2 Préparation des données

```

1 # Création du tableau de contingence
2 data = {

```

```

3      'Cheveux': ['Noir', 'Noir', 'Noir', 'Noir',
4      'Brun', 'Brun', 'Brun', 'Brun',
5      'Roux', 'Roux', 'Roux', 'Roux',
6      'Blond', 'Blond', 'Blond', 'Blond'],
7      'Yeux': ['Brun', 'Bleu', 'Noisette', 'Vert'] * 4,
8      'Effectif': [68, 20, 15, 5,
9      119, 84, 54, 29,
10     26, 17, 14, 14,
11     7, 94, 10, 16]
12 }
13
14 df = pd.DataFrame(data)
15
16 # Transformation en tableau de contingence
17 table = pd.crosstab(
18     index=pd.Categorical(df['Cheveux'],
19     categories=['Noir', 'Brun', 'Roux', 'Blond']),
20     columns=pd.Categorical(df['Yeux'],
21     categories=['Brun', 'Bleu', 'Noisette', 'Vert']),
22     values=df['Effectif'],
23     aggfunc='sum'
24 ).fillna(0)
25
26 print("Tableau de contingence:")
27 print(table)
28 print("\nTotal:", table.values.sum())

```

3.3 Réalisation de l'AFC

```

1      # Méthode 1: Utilisation de prince (spécialisé pour l'AFC)
2      ca = prince.CA(
3      n_components=2,
4      n_iter=3,
5      copy=True,
6      check_input=True,
7      engine='sklearn',
8      random_state=42
9      )
10
11      ca = ca.fit(table)
12
13      # Coordonnées
14      row_coords = ca.row_coordinates(table)
15      col_coords = ca.column_coordinates(table)
16
17      print("Coordonnées des lignes (Cheveux):")
18      print(row_coords)
19
20      print("\nCoordonnées des colonnes (Yeux):")
21      print(col_coords)
22
23      # Valeurs propres
24      eigenvalues = ca.eigenvalues_
25      print(f"\nValeurs propres: {eigenvalues}")
26      print(f"Inertie totale: {sum(eigenvalues):.4f}")

```

```

27 print(f"Inertie expliquée Dim1: {eigenvalues[0]/sum(eigenvalues)
    *100:.2f}%")
28 print(f"Inertie expliquée Dim2: {eigenvalues[1]/sum(eigenvalues)
    *100:.2f}%")

```

3.4 Visualisation des résultats

```

1  # Graphique de l'éboulis des valeurs propres
2  fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
3
4  # Éboulis
5  inertia_exp = eigenvalues / sum(eigenvalues) * 100
6  cum_inertia = np.cumsum(inertia_exp)
7
8  ax1.bar(range(1, len(eigenvalues)+1), inertia_exp, alpha=0.7)
9  ax1.plot(range(1, len(eigenvalues)+1), cum_inertia,
10 'ro-', linewidth=2)
11 ax1.set_xlabel('Composantes')
12 ax1.set_ylabel('Pourcentage d\'inertie')
13 ax1.set_title('Éboulis des valeurs propres')
14 ax1.grid(True, alpha=0.3)
15
16 for i, (v, c) in enumerate(zip(inertia_exp, cum_inertia)):
17     ax1.text(i+1, v+1, f'{v:.1f}%', ha='center')
18     ax1.text(i+1, c+1, f'{c:.1f}%', ha='center')
19
20 # Carte factorielle
21 ax2.scatter(row_coords[0], row_coords[1],
22             alpha=0.7, s=100, c='blue', label='Cheveux')
23 ax2.scatter(col_coords[0], col_coords[1],
24             alpha=0.7, s=100, c='red', label='Yeux')
25
26 # Étiquettes
27 for idx, row in row_coords.iterrows():
28     ax2.annotate(idx, (row[0], row[1]),
29                 xytext=(5, 5), textcoords='offset points')
30
31 for idx, row in col_coords.iterrows():
32     ax2.annotate(idx, (row[0], row[1]),
33                 xytext=(5, 5), textcoords='offset points')
34
35 ax2.axhline(y=0, color='grey', linestyle='--', alpha=0.5)
36 ax2.axvline(x=0, color='grey', linestyle='--', alpha=0.5)
37 ax2.set_xlabel(f'Dimension 1 ({inertia_exp[0]:.1f}%)')
38 ax2.set_ylabel(f'Dimension 2 ({inertia_exp[1]:.1f}%)')
39 ax2.set_title('Carte factorielle AFC')
40 ax2.legend()
41 ax2.grid(True, alpha=0.3)
42
43 plt.tight_layout()
44 plt.show()
45
46 # Visualisation avec prince
47 ax = ca.plot_coordinates(
48 X=table,

```

```

49     ax=None,
50     figsize=(10, 8),
51     x_component=0,
52     y_component=1,
53     show_row_labels=True,
54     show_col_labels=True
55 )
56 ax.set_title('AFC - Carte factorielle (prince)', fontsize=16)
57 plt.show()

```

3.5 Calcul des contributions et cosinus carrés

```

1     # Calcul manuel des contributions
2     def calculate_contributions(table, row_coords, col_coords,
3                                eigenvalues):
4         """Calcule les contributions et cos2"""
5
6         # Tableau des profils lignes
7         row_profiles = table.div(table.sum(axis=1), axis=0)
8
9         # Tableau des profils colonnes
10        col_profiles = table.div(table.sum(axis=0), axis=1)
11
12        # Masses (poids) des lignes et colonnes
13        row_masses = table.sum(axis=1) / table.sum().sum()
14        col_masses = table.sum(axis=0) / table.sum().sum()
15
16        # Contributions des lignes
17        row_contrib = pd.DataFrame(
18            row_masses.values.reshape(-1, 1) * row_coords.values**2 /
19            eigenvalues,
20            index=row_coords.index,
21            columns=['Dim1', 'Dim2']
22        )
23
24        # Contributions des colonnes
25        col_contrib = pd.DataFrame(
26            col_masses.values.reshape(-1, 1) * col_coords.values**2 /
27            eigenvalues,
28            index=col_coords.index,
29            columns=['Dim1', 'Dim2']
30        )
31
32        return row_contrib, col_contrib, row_masses, col_masses
33
34        row_contrib, col_contrib, row_masses, col_masses =
35            calculate_contributions(
36                table, row_coords, col_coords, eigenvalues
37            )
38
39        print("Contributions des lignes (%):")
40        print((row_contrib * 100).round(2))
41
42        print("\nContributions des colonnes (%):")
43        print((col_contrib * 100).round(2))

```

4 Interprétation des résultats

4.1 Résultats principaux

- **Inertie totale** : 0.2634
- **Dim 1** : 79.24% de l'inertie (associée à l'opposition cheveux foncés/yeux bruns vs cheveux blonds/yeux bleus)
- **Dim 2** : 16.80% de l'inertie (associée aux couleurs rousses/vert)

4.2 Contributions importantes

Variable	Contribution Dim1	Contribution Dim2
Lignes		
Blond	68.2%	1.4%
Brun	16.3%	44.6%
Noir	12.9%	1.8%
Roux	2.6%	52.2%
Colonnes		
Bleu	67.5%	0.9%
Brun	20.1%	45.9%
Vert	6.4%	49.5%
Noisette	6.0%	3.7%

TABLE 2 – Contributions principales à l'inertie

4.3 Associations révélées

1. **Axe 1** : Opposition entre :
 - **Pôle positif** : Cheveux blonds (0.96) et Yeux bleus (0.85)
 - **Pôle négatif** : Cheveux noirs (-0.70), bruns (-0.24) et Yeux bruns (-0.60)
2. **Axe 2** : Opposition entre :
 - **Pôle positif** : Cheveux roux (0.43) et Yeux verts (0.33)
 - **Pôle négatif** : Cheveux bruns (-0.26) et Yeux bruns (-0.23)

5 Conclusion

L'AFC a permis de visualiser clairement les associations entre couleurs de cheveux et couleurs des yeux :

- Les cheveux blonds sont fortement associés aux yeux bleus
- Les cheveux bruns et noirs sont associés aux yeux bruns
- Les cheveux roux sont associés aux yeux verts
- Les yeux noisette n'ont pas d'association spécifique forte

Ces résultats montrent l'utilité de l'AFC pour explorer les relations entre variables catégorielles et faciliter l'interprétation des tableaux de contingence complexes.

A Annexe : Théorie de l'AFC

L'AFC repose sur la décomposition en valeurs singulières de la matrice des écarts à l'indépendance :

$$X = \frac{p_{ij} - p_{i.}p_{.j}}{\sqrt{p_{i.}p_{.j}}}$$

où :

- $p_{ij} = \frac{n_{ij}}{n}$ proportion conjointe
- $p_{i.} = \frac{n_{i.}}{n}$ marge ligne
- $p_{.j} = \frac{n_{.j}}{n}$ marge colonne

La distance du χ^2 entre profils lignes s'écrit :

$$d^2(i, i') = \sum_{j=1}^p \frac{1}{p_{.j}} \left(\frac{p_{ij}}{p_{i.}} - \frac{p_{i'j}}{p_{i'.}} \right)^2$$

B Références

- Benzécri, J.P. (1973). *L'Analyse des Données*
- Lebart, L., Morineau, A., & Piron, M. (1995). *Statistique exploratoire multidimensionnelle*
- Husson, F., Lê, S., & Pagès, J. (2017). *Analyse de données avec R*